

Progettazione e Realizzazione di un Veicolo da Trasporto Teleoperato in CANopen

Andrea Efficace

`andrea.efficace@alumni.uniroma2.eu`

`a.efficace@daikinapplied.eu`

Università degli Studi di Roma Tor Vergata
Macroarea di Ingegneria
Corso di Laurea Magistrale in Ingegneria dell'Automazione

Anno Accademico 2024/2025



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA



DAIKIN

- 1 Introduzione
- 2 Stato dell'arte: robot mobili, navigazione e protocolli
- 3 Requisiti e Componenti
- 4 Modellazione Cinematica e Controllo
- 5 Architettura Software e Comunicazione
- 6 Conclusioni e Sviluppi Futuri

- 1 Introduzione
- 2 Stato dell'arte: robot mobili, navigazione e protocolli
- 3 Requisiti e Componenti
- 4 Modellazione Cinematica e Controllo
- 5 Architettura Software e Comunicazione
- 6 Conclusioni e Sviluppi Futuri

Contesto Aziendale: Daikin Applied Europe

- **Daikin Applied Europe (DAE):**
 - ▶ Sede di Ariccia (Roma).
- Leader nel settore HVAC (Heating, Ventilation, Air Conditioning).
- Produzione **Make-to-Order:**
 - ▶ Unità Trattamento Aria (UTA) e Chiller.
 - ▶ Prodotti personalizzati per il mercato europeo.
- **Driver:** Crescente domanda per soluzioni in ambiti industriali rilevanti nel momento storico attuale (es: datacenter).



Obiettivo: Ottimizzazione dei processi produttivi verso l'**Industria 5.0**.

Progetti Chiave

- **MES (Manufacturing Execution System):** Monitoraggio produzione in tempo reale.
- **Test-bench automatici:** Collaudo sottoassiemi e prodotti finiti.
- **Visione Artificiale:** Controllo qualità e ispezione.
- **Robotica Mobile:** Integrazione robot per movimentazione materiali.

Attualmente la movimentazione è manuale (muletti, carrelli), con costi elevati in termini di tempo e risorse.

Sfide della Produzione Make-to-Order

- **Percorsi Variabili:** Destinazioni cambiano in base alle esigenze giornaliere.
- **Layout Dinamico:** Disposizione spazi variabile per necessità logistiche.
- **Ostacoli Dinamici:** Presenza costante di operatori e altri mezzi.

Progettazione e realizzazione di un **Veicolo Autonomo (AMR)** per lo stabilimento di Ariccia.

- **Dimostratore Tecnologico:**

- ▶ Realizzazione di un prototipo a **basso costo**.
- ▶ Valutazione fattibilità in ambiente industriale reale, all'aperto e poco strutturato.

- **Transizione all'Automazione:**

- ▶ Sostituzione movimentazione manuale.
- ▶ Integrazione futura con sistemi **WMS** (Warehouse Management System) ed **ERP** (Enterprise Resource Planning).

- 1 Introduzione
- 2 Stato dell'arte: robot mobili, navigazione e protocolli
- 3 Requisiti e Componenti
- 4 Modellazione Cinematica e Controllo
- 5 Architettura Software e Comunicazione
- 6 Conclusioni e Sviluppi Futuri

AMR vs AGV

- **AGV**: Percorsi fissi, infrastruttura dedicata, bassa flessibilità.
- **AMR**: Navigazione autonoma, adattabilità, nessuna infrastruttura.

Tipologie di Locomozione

- **Ruote**: Differential drive, omnidirezionali.
- **Cingoli**: Terreni irregolari.
- **Zampe**: Ambienti complessi.
- **Volanti**: Droni/UAV.

Path Planning (Globale)

- **A***: Ricerca ottima su griglia.
- **Dijkstra**: Ricerca esaustiva.
- **RRT**: Esplorazione probabilistica.

Obstacle Avoidance (Locale)

- **VFH**: Istogramma polare ostacoli.
- **DWA**: Ottimizzazione velocità (v , ω).
- **Potential Fields**: Campi di forze.

Localizzazione

Tecniche per stimare la posizione del robot nell'ambiente.

- **Odometry**: Basata su encoder e IMU, soggetta a deriva.
- **Beaconing**: Utilizzo di segnali esterni (es. RFID).
- **SLAM**: Simultaneous Localization and Mapping.

Problema: Stimare posa e mappa simultaneamente in assenza di GPS.

Perché FAST-LIO 2?

Algoritmo LiDAR-Inertial Odometry (LIO) allo stato dell'arte^a.

- **Robustezza:** Funziona in ambienti spogli (corridoi lunghi) grazie alla *registrazione diretta* (no feature extraction).
- **Efficienza:** Struttura **ikd-Tree** per aggiornamento mappa veloce (> 100 Hz).
- **Accuratezza:** Fusione sensoriale (tightly-coupled) LiDAR + IMU.
- **Testato:** Già validato sul sensore acquistato (Livox Mid-360).

^aXu et al. "FAST-LIO2: Fast Direct LiDAR-Inertial Odometry", 2022;

Protocolli di Comunicazione: CAN Bus

Controller Area Network (CAN): bus seriale multi-master per autoveicoli.

Caratteristiche Principali

- **Architettura Multi-Master:** Tutti i nodi possono trasmettere.
- **Arbitraggio Non-Distruttivo:** Priorità basata su ID messaggio.
- **Rilevamento Errori:** CRC, bit stuffing, monitoraggio ACK.
- **Velocità:** Fino a 1 Mbps su brevi distanze.

Applicazioni Industriali

Utilizzato in automotive, automazione industriale, robotica mobile.

CANopen: protocollo applicativo su CAN per automazione industriale.

Componenti Principali

- **Object Dictionary (OD):** dizionario parametri dispositivo.
- **PDO:** messaggi real-time (velocità, encoder).
- **SDO:** configurazione parametri (client-server).
- **NMT:** gestione stati nodi.

Estensione: Profilo CiA 402

- Standard per azionamenti elettrici
- Macchina a stati
- Modalità: Velocity, Position, Torque

- 1 Introduzione
- 2 Stato dell'arte: robot mobili, navigazione e protocolli
- 3 Requisiti e Componenti**
- 4 Modellazione Cinematica e Controllo
- 5 Architettura Software e Comunicazione
- 6 Conclusioni e Sviluppi Futuri

Selezione Payload

Payload: Compressore a Vite

- **Oggetto:** Compressore Daikin
- **Peso:** ~730 kg.
- **Dimensioni:** 1.9 × 0.9 × 0.75 m.
- **Logistica:** Trasporto da produzione a assemblaggio chiller.

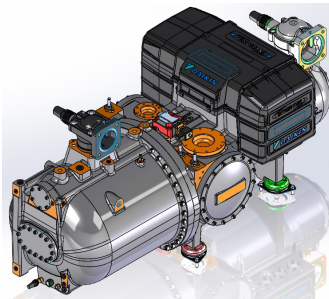


Figura 1: Compressore 35-A-CO1005

Scelta Progettuale

Dimensionamento sovrastimato a **1000 kg** (robot + payload) per scalabilità futura e margine di sicurezza.

Requisiti Veicolo

- **Capacità Carico:** 1000 kg.
- **Velocità Max:** 1 m/s (sicurezza).
- **Autonomia:** 4 ore operative.
- **Ambiente:** Indoor/Outdoor (asfalto, cemento).
- **Manovrabilità:** Omnidirezionale (parcheggio parallelo).
- **Economico:** Costo contenuto per dimostratore tecnologico.

Calcolo forza di trazione necessaria ($m = 1000 \text{ kg}$, $v_{max} = 1 \text{ m/s}$):

Parametri: $\mu_r = 0.03$ (attrito volvente), $\mu_s = 0.5$ (attrito statico), $a = 0.333 \text{ m/s}^2$

In Piano:

$$F_r = \mu_r \cdot m \cdot g = 294 \text{ N}$$

$$F_a = m \cdot a = 333 \text{ N}$$

$$F_{tot} = 627 \text{ N}$$

$$P_{piano} \approx 627 \text{ W}$$

Pendenza 10% ($\theta = 5.7^\circ$):

$$F_{\parallel} = mg \sin \theta = 974 \text{ N}$$

$$F_r = \mu_r mg \cos \theta = 293 \text{ N}$$

$$F_a = 333 \text{ N}$$

$$F_{tot} = 1600 \text{ N}$$

$$P_{salita} \approx 1.6 \text{ kW}$$

Selezione Batteria

FlashBattery LiFePO4 (Litio-Ferro-Fosfato)

- **Tensione:** 51.2 V nominali (44-59.2 V).
- **Capacità:** 105 Ah.
- **Energia Totale:** 5.38 kWh.
- **Energia Disponibile** (80% DOD): **4.30 kWh**.
- **Corrente:** 105 A continua, 210 A picco.



Verifica Autonomia

$$E_{\text{disponibile}} = 4.30 \text{ kWh} > 4 \text{ kWh (requisito)}$$

Margine: 7.5%

Configurazione: **2 Ruote Motrici Sterzanti** (Motorwheels) + 2 Caster (diagonali).

Specifiche per ruota:

● Trazione:

- ▶ Potenza: 500 W (S2-60min)
- ▶ Coppia Nom.: 37 Nm
- ▶ Coppia Picco: 180 Nm
- ▶ Rapporto riduzione: 1:21

● Sterzo:

- ▶ Potenza: 300 W
- ▶ Coppia: 34 Nm
- ▶ Rapporto riduzione: 1:45.56

● Encoder: SIN/COS (FTS1)

Verifica Coppia

Raggio ruota: $r = 0.099 \text{ m}$

Piano:

$$\tau_{tot} = F_{tot} \cdot r \approx 62 \text{ Nm}$$

$$\tau_{ruota} = \tau_{tot}/2 \approx 31 \text{ Nm}$$

$$\tau_{nom} = 37 \text{ Nm} \Rightarrow \text{Margine } 18\%$$

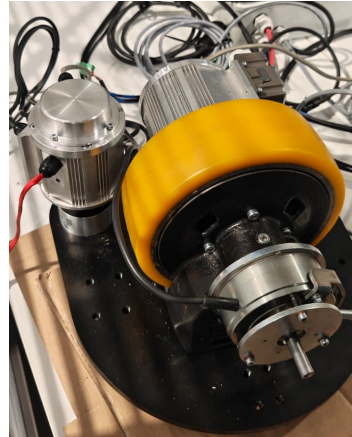
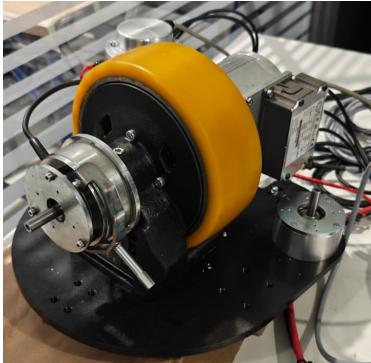
Salita 10%:

$$\tau_{tot} \approx 159 \text{ Nm}$$

$$\tau_{ruota} \approx 79.5 \text{ Nm}$$

$$\tau_{picco} = 180 \text{ Nm} \Rightarrow \text{OK}$$

Selezione Motori: CFR MRT05



- **Computer di Bordo:** LattePanda Sigma
 - ▶ Intel Core i5-1340P (12 core, 16 thread)
 - ▶ 32GB RAM LPDDR5-6400
 - ▶ Ubuntu 24.04 + ROS 2 Jazzy
- **LiDAR:** Livox Mid-360
 - ▶ FOV: $360^\circ \times 59^\circ$ (-7° a $+52^\circ$)
 - ▶ 200k pts/s, portata 70m
 - ▶ IMU 6-assi integrata (ICM40609)
- **Encoder:** SIN/COS (FTS1) integrati nelle motoruote
- **Sonde Temperatura:** RTD (STS1) per monitoraggio termico motori

Architettura Hardware: Alimentazione e Controllo

- **Driver Motori:** MicroPhase Mini-Traction Power (4x)
 - ▶ Supporto CANopen CiA 402
 - ▶ 48V, 75A continua, 120A boost (5s)
 - ▶ Controllo motori PMAC
- **Comunicazione:** Waveshare USB-CAN-A
 - ▶ CAN 2.0A/B, velocità fino a 1 Mbps
 - ▶ Resistenza terminazione 120Ω commutabile
- **Convertitori DC-DC:** Meanwell
 - ▶ SD-25B-12: 48V $\rightarrow$ 12V, 25W (LiDAR)
 - ▶ SD-100D-12: 48V $\rightarrow$ 12V, 102W (SBC)
- **Caricabatterie:** Zivan NG3
 - ▶ 48V, 60A configurabile, profilo CC-CV per LiFePO4
 - ▶ Comunicazione CAN con BMS, tempo ricarica: 2-3h

LattePanda Sigma



Livox Mid-360



**Driver Mini-Traction
Power**



USB-CAN-A Adapter

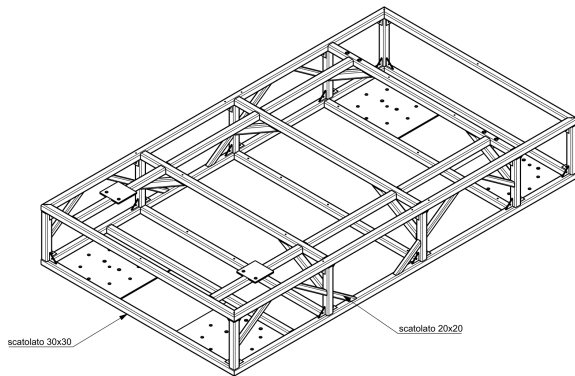


Caratteristiche:

- Massa: ~ 70 kg
- Profili acciaio $30 \times 30 \times 2.6$ mm
- Rinforzi $20 \times 20 \times 2$ mm
- Dimensioni: 1.8×1.0 m
- Piastre montaggio 5 mm
- Trattamento: zincatura

Progettazione:

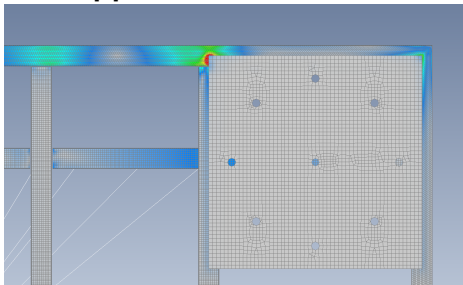
- CAD: SolidWorks
- Analisi FEM: FASTRAN



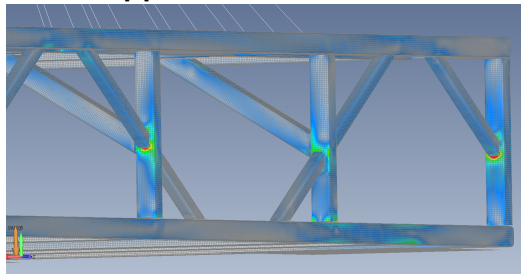
Analisi agli Elementi Finiti (FEM)

Verifica strutturale con software FASTRAN sotto carico 1000 kg.

Mappa delle Deformazioni



Mappa delle Deformazioni



Risultati

Deformazioni e tensioni entro limiti di sicurezza. Zone critiche localizzate nei punti di giunzione (artefatti mesh).

Componente	Costo (€)
Motoruote CFR MRT05 (2×)	5.480
Driver Mini-Traction Power (4×)	1.640
Batteria FlashBattery	3.950
Caricabatterie Zivan NG3	355
LattePanda Sigma	563
Livox Mid-360	659
Telaio, lamierati e caster	2.356
Cablaggi e messa in opera	1.800
Accessori (DC-DC, USB-CAN, SSD)	429
Totale	17.232

Considerazioni

Costo contenuto rispetto a soluzioni commerciali (ordine di grandezza inferiore).
Dimostratore tecnologico economico per validazione concetto.

- 1 Introduzione
- 2 Stato dell'arte: robot mobili, navigazione e protocolli
- 3 Requisiti e Componenti
- 4 Modellazione Cinematica e Controllo**
- 5 Architettura Software e Comunicazione
- 6 Conclusioni e Sviluppi Futuri

Modelli e Notazione Usata

Confrontati due modelli cinematici:

- **Biciclo Sterzante:** Preso dalla letteratura¹. Approssima un veicolo con quattro ruote sterzanti.
- **Doppia Ruota Sterzante:** proposta basata su vincoli non-olonomi, senza approssimazioni geometriche.
- **Stato:** $q = [x \ y \ \theta]^T$,
- **Dimensioni:** $L, W, L > W$,
- **Posizioni delle Ruote:**
 $p_f^b = [L/2 \ W/2]^T$
 $p_r^b = -[L/2 \ W/2]^T$
- **Input di Controllo:**
 $u = [\omega_f \ \omega_r \ \delta_f \ \delta_r]^T$,
- **Velocità Angolare Ruote:** ω_f, ω_r ,
- **Angoli di Sterzo:** δ_f, δ_r ,
- **Raggio Ruote:** r .

¹Tourajizadeh et al., "Modeling and Optimal Control of 4 Wheel Steering Vehicle Using LQR", 2020; 

- **Vel. Centro Robot (body frame):** $\dot{x}_c^b, \dot{y}_c^b$
- **Assunzione:** simmetria geometrica
 - ▶ Centro geometrico coincide con centro di massa
 - ▶ Distanza tra centro e asse ruote: $l_f = l_r = L/2 \Rightarrow l_f + l_r = L$
- **Convenzioni:** angoli positivi in senso antiorario
 - ▶ θ zero quando asse longitudinale parallelo a asse x globale
 - ▶ **Sterzi** zero quando ruote parallele all'asse longitudinale
- **Velocità angolari positive:** rotazione in avanti (avanzamento)

Modello a Biciclo Sterzante

Semplificazione: ruote sullo stesso asse $\rightarrow$ ruota virtuale unica per asse

Equazioni Cinematiche

$$\begin{cases} \dot{x} = v \cos(\theta + \beta) \\ \dot{y} = v \sin(\theta + \beta) \\ \dot{\theta} = \frac{v \cos(\beta)}{l_f + l_r} (\tan(\delta_f) - \tan(\delta_r)) \end{cases} \quad (1)$$

dove:

$$v = \frac{r(\omega_f \cos \delta_f + \omega_r \cos \delta_r)}{2 \cos(\beta)} \quad (2)$$

$$\beta = \tan^{-1} \left(\frac{l_f \tan(\delta_r) + l_r \tan(\delta_f)}{l_f + l_r} \right) \quad (\text{side-slip}) \quad (3)$$

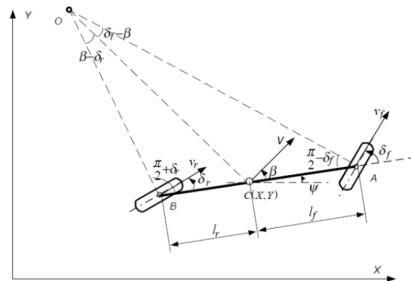


Figura 2: Modello a Biciclo Sterzante

Modello a Biciclo Sterzante: Osservazioni

Vantaggi

- Testato in letteratura.
- Modello dinamico studiato (controllabile con LQR).

Svantaggi: Singularità

- $\beta \rightarrow \infty$ se δ_f o $\delta_r \rightarrow \pi/2$ (tangente infinita)
- **Crab/Spin impossibile:** se $v = 0$ allora $\omega_f = \omega_r = 0 \Rightarrow \dot{\theta} = 0$

Modello a Doppia Ruota Sterzante

Soluzione finale nel body frame:

$$\begin{bmatrix} \dot{x}_c^b \\ \dot{y}_c^b \\ \dot{\theta} \end{bmatrix} = \frac{r}{2} \begin{bmatrix} \cos(\delta_f) & \cos(\delta_r) \\ \sin(\delta_f) & \sin(\delta_r) \\ a_{13} & a_{23} \end{bmatrix} \begin{bmatrix} \omega_f \\ \omega_r \end{bmatrix} \quad (4)$$

dove i coefficienti a_{13} , a_{23} dipendono da δ_f , δ_r , L , W :

$$a_{13} = \frac{2(L \sin(\delta_f) - W \cos(\delta_f))}{L^2 + W^2} \quad (5)$$

$$a_{23} = \frac{2(W \cos(\delta_r) - L \sin(\delta_r))}{L^2 + W^2} \quad (6)$$

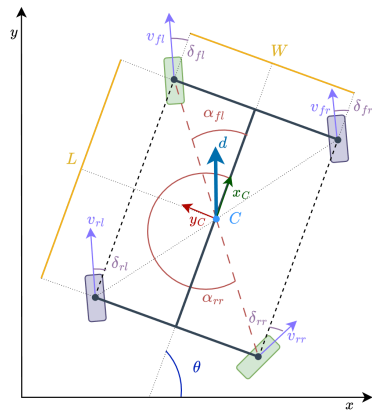


Figura 3: Ruote motrici in verde, passive in viola.

Modello a Doppia Ruota Sterzante: Osservazioni

Vantaggi

- **Nessuna singolarità cinematica** per qualsiasi δ_f, δ_r .
- Gestisce **crab** ($\dot{x}_c = 0, \dot{y}_c \neq 0$) e **spin** ($\dot{x}_c = \dot{y}_c = 0, \dot{\theta} \neq 0$)

Svantaggi

- No modellazione dinamica (forze, inerzie).
- No modellazione ruote libere.
- Soluzione ai minimi quadrati: possibilità di violare vincoli non-olonomi.

Parametri Robot e Attuatori

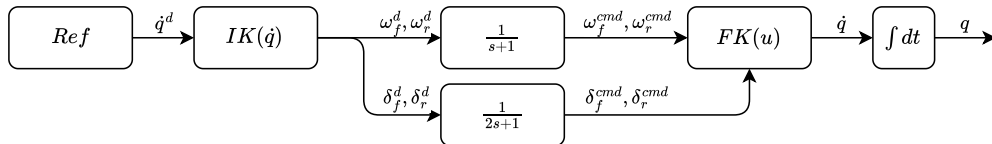
- Dimensioni: $L = 2 \text{ m}$, $W = 1 \text{ m}$, $d = 0.198 \Rightarrow r = 0.099 \text{ m}$
- Dinamica attuatori: modello 1° ordine $P(s) = \frac{K}{\tau s + 1}$
 - ▶ Trazione: $K_\omega = 1$, $\tau_\omega = 1 \text{ s}$, $\omega_{max} = 10 \text{ rad/s}$
 - ▶ Sterzo: $K_\delta = 1$, $\tau_\delta = 2 \text{ s}$, $\delta_{max} = \pm 120^\circ$

Traiettorie di Test

- 1 **Straight**: linea retta verso Est ($v_{ref} = 0.5 \text{ m/s}$, $T = 5 \text{ s}$)
- 2 **Arc**: arco circolare ($R = 3 \text{ m}$, $v_{ref} = 0.6 \text{ m/s}$, $\omega = 0.2 \text{ rad/s}$, $T = 8 \text{ s}$)
- 3 **Crab**: traslazione laterale Nord ($v_{ref} = 0.4 \text{ m/s}$, $T = 5 \text{ s}$)
- 4 **Spin**: rotazione sul posto ($\omega_{ref} = \pi/6 \text{ rad/s}$, $T = 6 \text{ s}$)

Metrica: RMSE posizione e orientamento vs. traiettoria desiderata

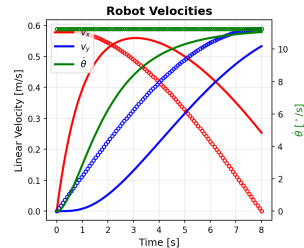
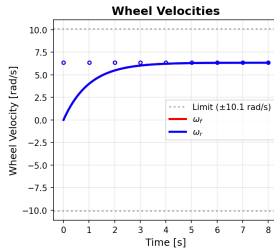
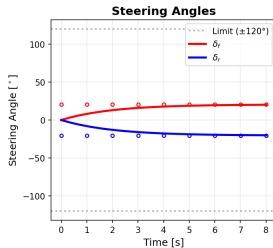
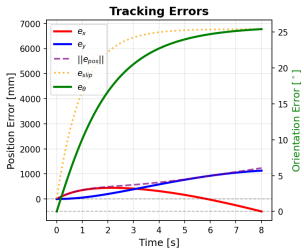
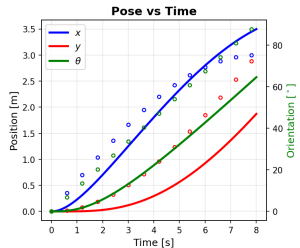
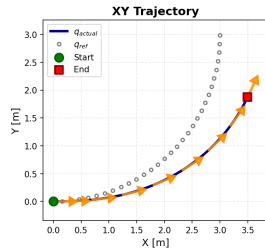
Schema di Controllo: Feedforward (Catena Aperta)



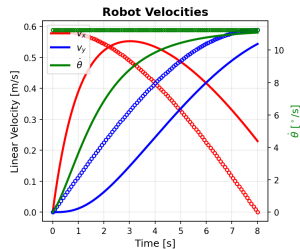
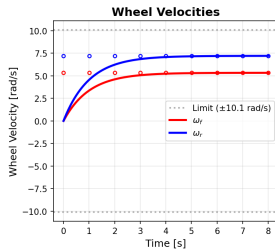
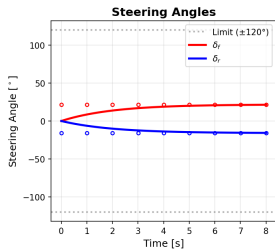
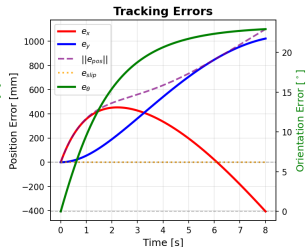
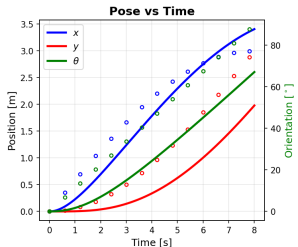
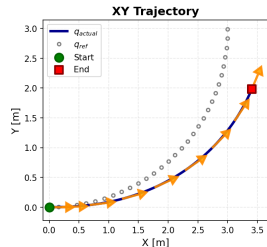
Caratteristiche:

- Nessun feedback dallo stato misurato
- Cinematica inversa calcola input da traiettoria desiderata
- Dinamica attuatori introduce ritardi non compensati
- Errori si accumulano senza correzione

Risultati Feedforward: Traiettoria Arc – Biciclo



Risultati Feedforward: Traiettoria Arc – Doppio Sterzo

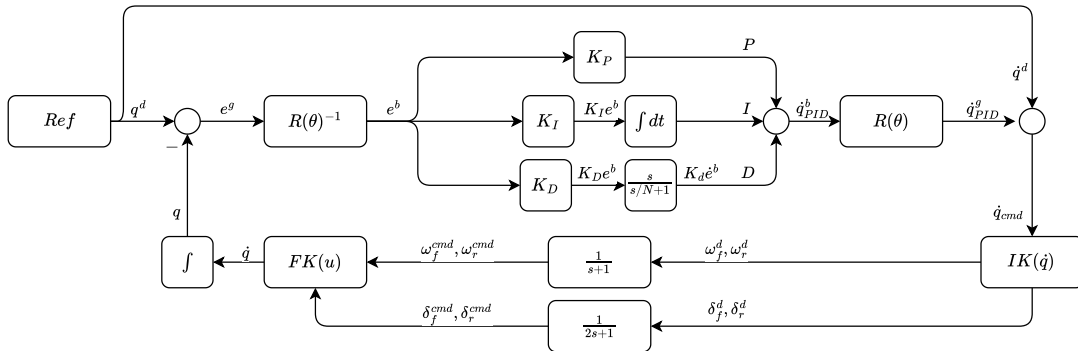


Traiett.	Mod.	Supp.	Pos. [mm]	Orient. [°]
Str.	Bic.	x	418.05	0.00
	Bis.	x	418.05	0.00
Arc	Bic.	x	749.92	20.80
	Bis.	x	694.70	18.85
Cra.	Bic.	–	1154.82	0.00
	Bis.	x	334.44	0.00
Spi.	Bic.	–	0.00	103.93
	Bis.	x	0.00	25.93

Osservazioni:

- Biciclo FALLISCE su Crab e Spin (singolarità)
- Doppio Sterzo gestisce tutte le manovre
- Errori elevati per assenza di feedback

Schema di Controllo: PID + Feedforward



Legge di controllo:

$$e^b = R^{-1}(\theta) \begin{bmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{bmatrix} \quad (\text{errore in body frame}) \quad (7)$$

$$\dot{q}_{i_{\text{PID}}}^b(t) = K_{p,i} e_i^b + K_{i,i} \int e_i^b d\tau + K_{d,i} \frac{de_i^b}{dt}, \quad i \in \{x, y, \theta\} \quad (8)$$

$$\dot{q}_{\text{cmd}} = \dot{q}^d + R(\theta) \dot{q}_{i_{\text{PID}}}^b \quad (9)$$

$$\mathbf{u} = \mathbf{IK}(\dot{q}_{\text{cmd}}) \quad (10)$$

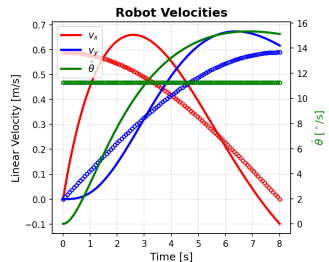
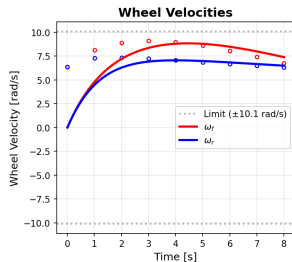
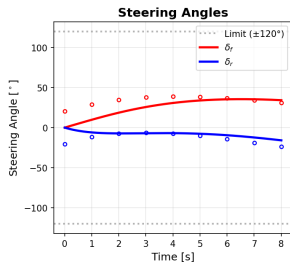
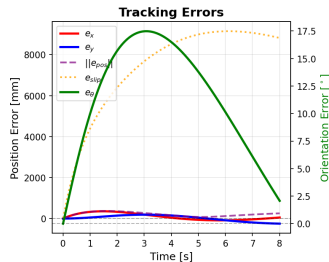
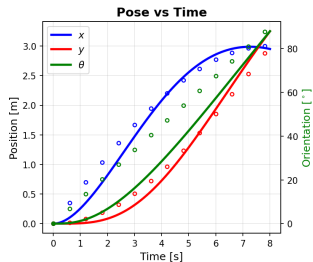
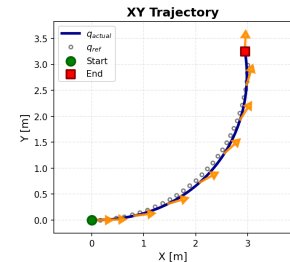
Guadagni utilizzati:

Comp.	K_p	K_i	K_d	I_{max}	N
x, y	0.3	0.05	0.1	2	20
θ	0.2	0.02	0.05	1	20

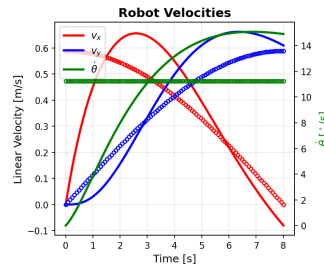
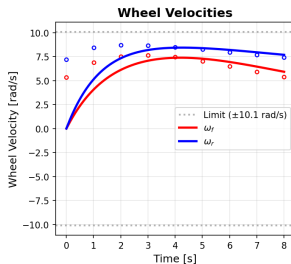
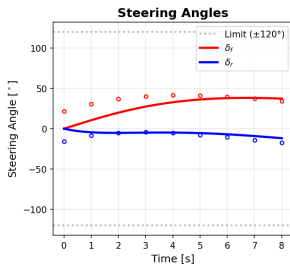
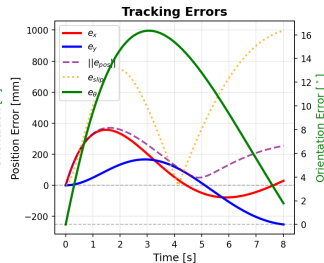
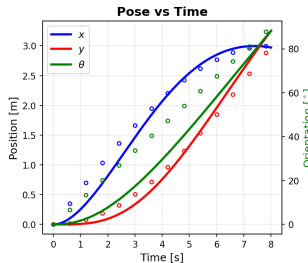
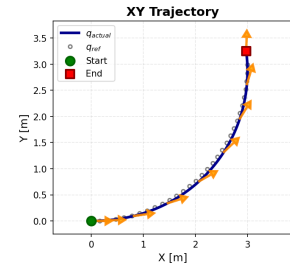
Accorgimenti implementativi:

- **Derivata filtrata:** $D(s) = \frac{K_d s}{s/N+1}$ (riduce rumore)
- **Anti-windup:** saturazione integrale per evitare wind-up
- **Scaling proporzionale:** mantiene rapporto tra u_x, u_y sotto v_{max}
- **Saturazione angolare:** $|u_\theta| \leq \omega_{max}$

Risultati PID+FF: Traiettoria Arc – Biciclo



Risultati PID+FF: Traiettoria Arc – Doppio Sterzo



Risultati PID+FF: Confronto e Miglioramenti

Traiett.	Mod.	Supp.	Pos. [mm]	Orient. [°]
Str.	Bic.	x	233.96	0.00
	Bis.	x	233.96	0.00
Arc	Bic.	x	231.27	12.53
	Bis.	x	229.57	11.66
Cra.	Bic.	–	1154.82	0.00
	Bis.	x	187.17	0.00
Spi.	Bic.	–	0.00	103.93
	Bis.	x	0.00	16.14

Miglioramenti vs FF:

- Str: 418 → 234 mm
- Arc: 695 → 230 mm
- Crab: 334 → 187 mm
- Spin: 26° → 16°

Conclusioni: Confronto Modelli Cinematici

Modello Biciclo

- + Testato in letteratura
- + Estensione dinamica disponibile
- **Singolarità**: Crab e Spin impossibili
- $\beta \rightarrow \infty$ per $\delta \rightarrow \pi/2$

Modello Doppio Sterzo

- + **Nessuna singolarità**
- + Supporta tutte le manovre
- + Prestazioni migliori con PID
- ! Pseudoinversa può violare vincoli

Scelta finale: **Modello Doppio Sterzo** per maggiore versatilità.

- 1 Introduzione
- 2 Stato dell'arte: robot mobili, navigazione e protocolli
- 3 Requisiti e Componenti
- 4 Modellazione Cinematica e Controllo
- 5 Architettura Software e Comunicazione**
- 6 Conclusioni e Sviluppi Futuri

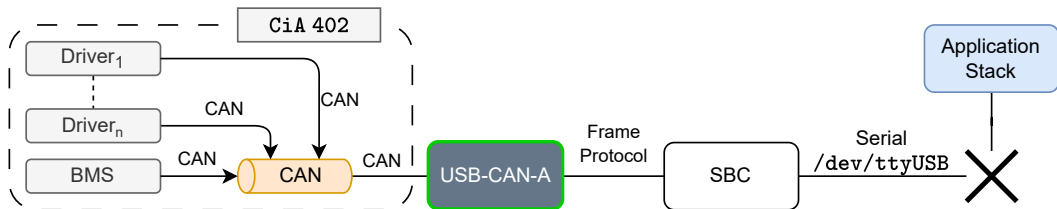
Problema di Integrazione Hardware-Software

Vincolo Hardware: Waveshare USB-CAN-A usa **protocollo seriale proprietario**

- Non compatibile con e.g. SocketCAN Linux, implementazioni CANopen.
- Documentato.

Requisito Software: Integrazione con ROS 2 per controllo robot

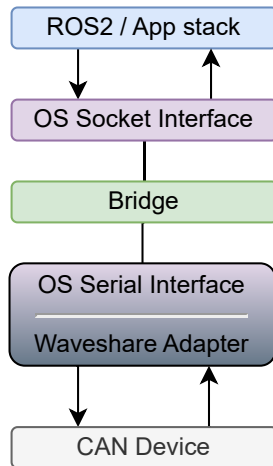
- Per sviluppo algoritmi di navigazione e controllo (es *FastLIO2*, *Nav2*, *teleop*).
- Necessità comunicazione real-time con inverter motori.



Architettura Software: SocketCAN Bridge

Bridge: Architettura a livelli per integrazione trasparente

- 1 **ROS2 / APP Layer:** Controllo, SLAM, navigazione.
- 2 **SocketCAN:** Interfaccia CAN nativa in kernel Linux.
- 3 **Bridge Software:** Traduzione bidirezionale tra SocketCAN e Frame Protocol (libreria `waveshare_cpp`).
- 4 **Waveshare USB-CAN-A:** Adattatore USB-CAN.
- 5 **CAN Bus:** Comunicazione con dispositivi CANopen.



Scopo: Gestione protocollo proprietario USB-CAN-A (Waveshare) per (de)serializzare frame CAN su USB.

Funzionalità principali:

- **Encoding/decoding frame CAN**

- ▶ Standard ID (11-bit) ed Extended ID (29-bit).
- ▶ Payload fino a 8 byte per frame.

- **Configurazione adapter e interfaccia seriale**

- ▶ Baud rate CAN bus (50 kbps – 1 Mbps).
- ▶ Filtri e maschere per ricezione ID-based.

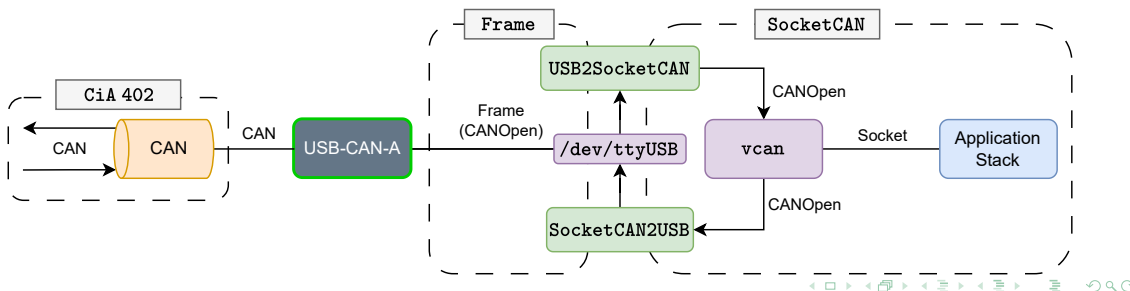
- **Gestione thread-safe**

- ▶ Accesso concorrente lettura/scrittura.
- ▶ Sincronizzazione automatica

Bridge USB ↔ SocketCAN

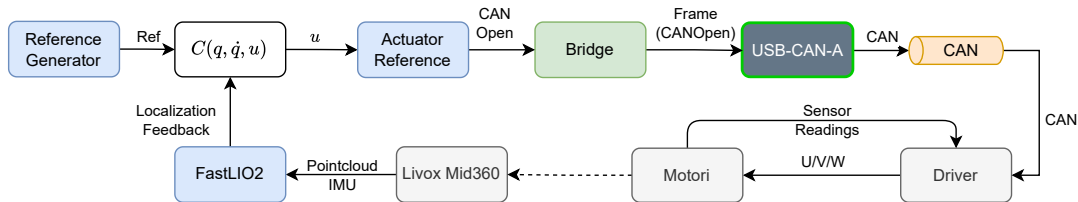
- Architettura a tre thread:

- ▶ **Thread principale:** Inizializza adapter e socket CAN virtuale, avvia thread di lettura/scrittura.
- ▶ **Thread di lettura:** Legge dati da USB, decodifica frame Waveshare estraendone i messaggi CAN e li invia alla socket.
- ▶ **Thread di scrittura:** Riceve frame CAN dalla socket, li codifica in un frame Waveshare e li invia a USB.



Stack Completo di Comunicazione e Controllo

- **Ref. Generator**: nodo ROS2 per generare riferimenti di velocità e sterzo.
- $C(q, \dot{q}, u)$: nodo ROS2 che implementa il controllo del modello cinematico (PID+FF).
- **Bridge + waveshare_cpp**: interfaccia tra ROS2 e hardware CANopen.
- **Driver + Motori**: controllo a basso livello.
- **LiDAR + SLAM**: mappatura e localizzazione.



- 1 Introduzione
- 2 Stato dell'arte: robot mobili, navigazione e protocolli
- 3 Requisiti e Componenti
- 4 Modellazione Cinematica e Controllo
- 5 Architettura Software e Comunicazione
- 6 Conclusioni e Sviluppi Futuri**

Rispetto ai requisiti iniziali, il progetto proposto ha raggiunto i seguenti obiettivi:

- Dimensionamento dei componenti meccanici ed elettrici rispetto alle specifiche di carico e prestazioni richieste.
- Selezione e acquisto dei materiali da diversi fornitori, acquisendo contatti per collaborazioni future.
- Progettazione e realizzazione di un dimostratore, con predisposizione per diversi layout di configurazione.
- Proposta di un'architettura di controllo per l'inseguimento di traiettorie.
- Sviluppo di un'architettura software per la comunicazione con gli inverter delle motoruote mediante adattatore USB, trasparente ad implementazioni CANOpen.

- Libreria di comunicazione con adattatore USB-CAN-A ampiamente testata.
- Architettura software definita e parzialmente implementata.
 - ▶ Mancanti: Nodi ROS2 per generazione traiettorie e invio comandi.
- Nodi ROS2 per l'avvio e la gestione dell'architettura di comunicazione, con pubblicazione dello stato dei motori e lettura dei sensori.
- Simulazione in Python dell'architettura di controllo per l'inseguimento di traiettorie.

- Acquisto di tutto l'hardware elencato completato.
- Proposto schema di cablaggio per l'integrazione dei sistemi su un banco di prova.
- Realizzazione del banco di prova per la validazione dei sistemi e dell'architettura software.



Figura 4: Banco di test per una motoruota.



Figura 5: Carrello e lamiera

1 Implementazione Hardware e Software

- ▶ Completare e integrare lo stack software completo su robot fisico,
- ▶ Assemblaggio e cablaggio dei componenti hardware.

2 Validazione Sperimentale

- ▶ Test su campo presso stabilimento Daikin Ariccia,
- ▶ Misurazione prestazioni: precisione tracking, autonomia, affidabilità,
- ▶ Ottimizzazione parametri controllo su dati reali,

3 Estensioni

- ▶ Fleet management per coordinamento multi-robot,
- ▶ Integrazione con MES aziendale per pianificazione trasporti,
- ▶ Sensori aggiuntivi: telecamere RGB-D, safety laser scanner.
- ▶ Test di altri modelli cinematici e/o dinamici e architetture di controllo avanzate.

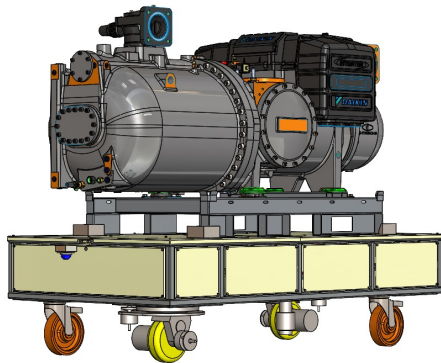


Figura 6: Mockup del veicolo progettato; nome in codice: Cocco

Appendice: IK Biciclo – Centro Inst. di Rotazione (ICR)

Concetto: punto attorno al quale il veicolo sta ruotando in un dato istante

L'ICR si trova all'**intersezione degli assi di rotazione delle ruote** (perpendicolari alla direzione di avanzamento):

$$\begin{bmatrix} x_{ICR} \\ y_{ICR} \end{bmatrix} = \mathbf{p}_f^b + s_f \begin{bmatrix} -\sin(\delta_f) \\ \cos(\delta_f) \end{bmatrix} = \mathbf{p}_r^b + s_r \begin{bmatrix} -\sin(\delta_r) \\ \cos(\delta_r) \end{bmatrix} \quad (11)$$

Risolvendo per s_f, s_r si trovano i raggi di curvatura delle ruote:

$$R_f = \sqrt{\left(x_{ICR} - \frac{L}{2}\right)^2 + (y_{ICR})^2}, \quad R_r = \sqrt{\left(x_{ICR} + \frac{L}{2}\right)^2 + (y_{ICR})^2} \quad (12)$$

Condizione di compatibilità (moto senza slittamento):

$$\dot{\theta} = r \frac{\omega_f}{R_f} = r \frac{\omega_r}{R_r} \Rightarrow R_r \omega_f = R_f \omega_r \quad (13)$$

Caso $\delta_f = \delta_r$: ICR $\rightarrow \infty$, traslazione pura con $\omega_f = \omega_r$

Appendice: IK Biciclo

Problema: dato $(\dot{x}_d, \dot{y}_d, \dot{\theta}_d)$, calcolare $(\delta_f, \delta_r, \omega_f, \omega_r)$

Passo 1: Calcolare velocità e side-slip desiderati

$$v_d = \sqrt{\dot{x}_d^2 + \dot{y}_d^2} \quad (14)$$

$$\beta_d = \text{atan2}(\dot{y}_d, \dot{x}_d) - \theta \quad (15)$$

Passo 2: Definendo $S = \tan(\beta_d)$, $D = \frac{l\dot{\theta}_d}{v_d \cos(\beta_d)}$, da (1) e (3):

$$\begin{cases} lS = l_f \tan(\delta_r) + l_r \tan(\delta_f) \\ D = \tan(\delta_f) - \tan(\delta_r) \end{cases} \Rightarrow \begin{cases} \delta_f = \arctan\left(S + \frac{l_f}{l}D\right) \\ \delta_r = \arctan\left(S - \frac{l_r}{l}D\right) \end{cases} \quad (16)$$

Passo 3: Calcolare ω_f, ω_r da raggi ICR R_f, R_r e condizione di compatibilità

Derivazione per traslazione pura

Quando $\dot{\theta}_d = 0$, l'ICR $\rightarrow \infty$ e le direzioni di avanzamento delle ruote diventano parallele.

Step 1: Imporre $\delta_f = \delta_r = \delta$ (sterzi paralleli)

Step 2: Dalla relazione del side-slip angle:

$$\beta_d = \tan^{-1} \left(\frac{l_f + l_r}{l_f + l_r} \tan(\delta) \right) = \delta \quad (17)$$

Quindi il side-slip angle coincide con l'angolo di sterzo quando le ruote sono parallele.

Calcolo angoli di sterzo e velocità

Step 3: Combinando $\beta_d = \delta$ con $\beta_d = \text{atan2}(\dot{y}_d, \dot{x}_d) - \theta$:

$$\delta_f = \delta_r = \delta = \text{atan2}(\dot{y}_d, \dot{x}_d) - \theta \quad (18)$$

Step 4: Le velocità angolari sono uguali (condizione di compatibilità per ICR $\rightarrow \infty$):

$$\omega_f = \omega_r = \frac{v_d}{r} = \frac{\sqrt{\dot{x}_d^2 + \dot{y}_d^2}}{r} \quad (19)$$

Appendice: IK Doppia Ruota Sterzante

Problema: dato $(\dot{x}_d, \dot{y}_d, \dot{\theta}_d)$, calcolare $(\delta_f, \delta_r, \omega_f, \omega_r)$

⇒ **Soluzione diretta dai vincoli di non-slittamento e puro rotolamento:**

Angoli di sterzo (dalle eq. non-slip):

$$\delta_f = \text{atan2}\left(\dot{y}_d + \dot{\theta}_d \frac{L}{2}, \dot{x}_d - \dot{\theta}_d \frac{W}{2}\right) \quad (20)$$

$$\delta_r = \text{atan2}\left(\dot{y}_d - \dot{\theta}_d \frac{L}{2}, \dot{x}_d + \dot{\theta}_d \frac{W}{2}\right) \quad (21)$$

Velocità angolari ruote (dalle eq. rotolamento):

$$\omega_f = \frac{(\dot{y}_d + \dot{\theta}_d \frac{L}{2}) \sin(\delta_f) + (\dot{x}_d - \dot{\theta}_d \frac{W}{2}) \cos(\delta_f)}{r} \quad (22)$$

$$\omega_r = \frac{(\dot{y}_d - \dot{\theta}_d \frac{L}{2}) \sin(\delta_r) + (\dot{x}_d + \dot{\theta}_d \frac{W}{2}) \cos(\delta_r)}{r} \quad (23)$$

Caso speciale $\dot{\theta}_d = 0$: $\delta_f = \delta_r = \text{atan2}(\dot{y}_d, \dot{x}_d)$, $\omega_f = \omega_r = \frac{v_d}{r}$